

Curso de Capacitação em Desenvolvimento Full Stack

Desenvolvimento Mobile com React Native — Módulo 06

Professor: Marcelo Matos

Acesse aqui nossas
redes sociais.



SUMÁRIO

Aula 01 — Recapitulação: A Jornada até o Mobile.....	5
Revisão dos Módulos Anteriores.....	5
Aula 02 — Introdução ao React Native.....	5
Objetivos da Aula.....	5
O que é React Native?.....	5
Arquitetura do React Native.....	5
React.js vs React Native.....	6
Configurando o Ambiente com Expo.....	6
SafeAreaView e ScrollView.....	6
Composição de Estilos com Arrays.....	7
Dados Declarados Fora do Componente.....	7
Primeiro Componente Mobile.....	7
Estrutura do Projeto.....	8
Projeto Demo em Sala.....	8
Referências.....	8
Aula 03 — Componentes e Estilos.....	9
Objetivos da Aula.....	9
O Projeto desta Aula.....	9
StyleSheet Avançado.....	9
Flexbox Mobile.....	10
Componentes Reutilizáveis.....	10
Sombras no Android e iOS.....	11
Operadores Úteis no React Native.....	11
Calculando Totais com filter + reduce.....	11
Ícones com @expo/vector-icons.....	12
React Native Paper.....	12
Imagens no React Native.....	13
Projeto Demo em Sala.....	14
Referências.....	15
Aula 04 — Navegação e Rotas.....	15
Objetivos da Aula.....	15
O Projeto desta Aula.....	15
Por que precisamos de Navegação?.....	15
Instalação do React Navigation.....	15

Tab Navigator.....	16
Stack Navigator.....	17
Drawer Navigator.....	17
Navegando entre telas.....	20
Formulário de Nova Transação.....	20
Estrutura de Arquivos para Navegação.....	22
Navegação Condicional.....	22
KeyboardAvoidingView — Teclado que Cobre Campos.....	24
Recebendo Dados via route.params com useEffect.....	25
Passando Callback entre Telas.....	25
Projeto Demo em Sala.....	26
Referências.....	26
Aula 05 — APIs e Armazenamento.....	27
Objetivos da Aula.....	27
Pré-Requisitos — Configure Antes de Começar.....	27
O Projeto desta Aula.....	27
AsyncStorage — Armazenamento Local.....	28
Context API — Estado Global.....	28
Estados de Carregamento e Erro.....	29
Excluir Transação com Confirmação.....	30
Custom Hooks — Reutilizando Lógica.....	30
Consumindo uma API REST.....	31
Projeto Demo em Sala.....	32
Referências.....	32
Aula 06 — Armazenamento com SQLite.....	33
Objetivos da Aula.....	33
Pré-Requisitos — Configure Antes de Começar.....	33
Conteúdo Teórico.....	33
Estrutura de arquivos ao final da aula.....	35
Projeto Demo em Sala.....	35
Referências.....	36
Aula 07 — Geolocalização e Mapas.....	36
Objetivos da Aula.....	36
Pré-Requisitos — Configure Antes de Começar.....	36
Conteúdo Teórico.....	36
Estrutura de arquivos ao final da aula.....	37

Projeto Demo em Sala.....	38
Observações sobre produção.....	38
Referências.....	39
Aula 08 — Publicação e Deploy.....	39
Objetivos da Aula.....	39
O app.json — Identidade do App.....	39
Ícone e Splash Screen.....	40
EAS Build — Compilação na Nuvem.....	40
Keystore — Assinatura do App.....	41
Google Play — Fluxo de Publicação.....	41
App Store — Fluxo de Publicação.....	41
OTA Updates — Atualizações sem Build.....	42
Checklist de Produção.....	42
Referências.....	43

AULA 01 — RECAPITULAÇÃO: A JORNADA ATÉ O MOBILE

Revisão dos Módulos Anteriores

React Native é a **evolução natural** da jornada Full Stack. Cada módulo anterior contribui diretamente:

Módulo	Tema	Conexão com React Native
M1	Lógica de Programação	<code>map()</code> , <code>filter()</code> , eventos, manipulação de listas (FlatList)
M2	Arquitetura da Informação	Hierarquia de componentes, estrutura de navegação
M3	Web — HTML, CSS, JavaScript	ES6+, <code>async/await</code> , <code>fetch</code> — reutilizados diretamente
M4	React (Front-end) 	~70% do conhecimento necessário — componentes, hooks, JSX, props, state
M5	Node.js e Express	APIs REST que o app vai consumir

■ **Mensagem-chave:** Se você domina React.js do Módulo 4, já tem 70% do caminho percorrido!

AULA 02 — INTRODUÇÃO AO REACT NATIVE

Objetivos da Aula

Entender o que é React Native e sua arquitetura (Bridge/JavaScriptCore), configurar o ambiente com Expo CLI e criar o primeiro app mobile.

O que é React Native?

React Native é uma biblioteca criada pela Meta (Facebook) que permite criar aplicativos móveis **nativos** para iOS e Android usando JavaScript e React.

Por que “Native”?

Diferente de PWAs ou Cordova, o React Native **não usa webview**. O código JavaScript é convertido para componentes nativos reais:

- iOS → **UIView**, **UIButton**, etc. (Swift/Objective-C)
- Android → **ViewGroup**, **Button**, etc. (Kotlin/Java)

Resultado: performance próxima à de apps nativos puros.

Arquitetura do React Native

O React Native funciona em **3 camadas**: JavaScript Layer, BRIDGE, Native Layer (iOS / Android)

Como funciona na prática?

Você escreve:

```
<View>
  <Text>Olá, ITEAM!</Text>
</View>
```

A Bridge: 1. Recebe a instrução do JavaScript 2. Serializa: { type: 'createView', props: {...} } 3. Envia para a camada nativa 4. iOS cria um UIView com um UILabel dentro 5. Android cria um ViewGroup com um TextView dentro

Analogia: A Bridge é como um tradutor simultâneo trabalhando em segundo plano.

React.js vs React Native

O que MUDA (pequenas adaptações)

React Web	React Native	Observação
<code><div></code>	<code><View></code>	Container principal
<code> / <p></code>	<code><Text></code>	Todo texto deve estar dentro de <Text>
CSS (arquivo externo)	<code>StyleSheet.create({})</code>	Objeto JavaScript
<code>onClick</code>	<code>onPress</code>	Evento de toque
<code>import from 'react-dom'</code>	<code>import from 'react-native'</code>	Mudança de fonte

Configurando o Ambiente com Expo

Por que Expo? Expo simplifica drasticamente o setup. Você não precisa de Xcode ou Android Studio para começar — basta seu celular.

Aulas 2–4: podem ser feitas inteiramente no snack.expo.dev (https://snack.expo.dev) — sem instalar nada. Mesmo assim, **comece a instalar o ambiente local durante estas aulas:** a partir da Aula 5 ele se torna obrigatório.

Softwares necessários

Software	Versão	Download
Node.js	20.x LTS	nodejs.org (https://nodejs.org)
VS Code	qualquer recente	code.visualstudio.com (https://code.visualstudio.com)
Git	2.x	git-scm.com (https://git-scm.com)
Android Studio	2025.x	developer.android.com/studio (https://developer.android.com/studio)
Expo Go (celular)	atual	expo.dev/go (https://expo.dev/go)

Atividade prática: Siga o passo a passo no conteúdo complementar para criar e rodar o primeiro app durante a aula — do create-expo-app até o contador interativo com useState.

SafeAreaView e ScrollView

Em celulares modernos o conteúdo pode ficar escondido atrás da barra de status, da “franjinha” (notch) ou da Dynamic Island. O SafeAreaView garante que tudo fique dentro da área visível:

```
import { SafeAreaView, ScrollView } from 'react-native';

<SafeAreaView style={{ flex: 1, backgroundColor: '#f2f4f7' }}>
  <ScrollView contentContainerStyle={{ padding: 20 }}>
    /* conteúdo aqui nunca fica escondido */
  </ScrollView>
```

```
</SafeAreaView>
```

Detalhe importante: o `ScrollView` tem duas props de estilo: - `style` → estiliza o “container” do scroll (a caixa externa) - `contentContainerStyle` → estiliza o conteúdo interno (padding, alinhamento)

Sem `ScrollView`, o conteúdo maior que a tela simplesmente é cortado.

Composição de Estilos com Arrays

Você pode passar um **array** de estilos — o último tem prioridade sobre os anteriores. Isso permite criar variações sem duplicar código:

```
// Base + variante condicional:
<TouchableOpacity
  style={[styles.botao, styles.botaoCinza]} // ← array de estilos
  onPress={() => setContador(contador - 1)}
>
  <Text style={styles.botaoTexto}>-</Text>
</TouchableOpacity>
```

É equivalente a fazer `{ ...styles.botao, ...styles.botaoCinza }` — as propriedades do segundo objeto sobrescrevem as do primeiro.

Dados Declarados Fora do Componente

Arrays e objetos **estáticos** (que nunca mudam) devem ser declarados fora da função do componente:

```
// ✅ Fora do componente – criado uma única vez
const COMPARACOES = [
  { web: '<div>', nativo: '<View>' },
  { web: '<p>', nativo: '<Text>' },
];

export default function App() {
  // ✅ Dentro do componente – recriado a cada render
  const [contador, setContador] = useState(0);
  // ...
}
```

Se colocados dentro, o React recriaria o array a cada renderização — desperdício desnecessário. Dados que mudam ficam dentro (como `useState`); dados fixos ficam fora.

Primeiro Componente Mobile

O primeiro componente usa `View`, `Text` e `StyleSheet` — a mesma estrutura de um componente React Web, com tags diferentes.

Elemento
Importação
Container

React Web
`react-dom`
`<div>`

React Native
`react-native`
`<View>`

Elemento

Texto

Estilos

Estrutura do componente

React Web

<p>,

arquivo CSS

igual

React Native

<Text>

StyleSheet.create({})

igual ✓

O código completo (incluindo o contador com `useState` e a tabela de comparação) está no conteúdo complementar.

Estrutura do Projeto

```
MeuPrimeiroApp/
├── app/                # Telas (Expo Router)
│   └── index.tsx       # Tela inicial
├── assets/             # Imagens, fontes
├── components/         # Componentes reutilizáveis
├── app.json            # Configurações (nome, ícone, splash)
└── package.json        # Dependências
```

Projeto Demo em Sala

O diretório `ola-iteam/` contém um projeto Expo pronto para rodar durante a aula.

Como rodar

Siga as instruções completas no conteúdo complementar — o tutorial cobre desde a criação do projeto até a versão final com contador e tabela comparativa.

O que o demo mostra

Seção

Cabeçalho

Card Contador

Card Web → Mobile

Card Arquitetura

Conceito demonstrado

View, Text, StyleSheet

`useState + onPress + TouchableOpacity`

`map()` em array, comparação visual de tags

As 3 camadas do React Native visualmente

Estrutura do projeto demo

```
ola-iteam/
├── App.js              # Todo o código da demo (1 arquivo só)
├── app.json            # Configuração Expo
├── babel.config.js     # Configuração do compilador
└── package.json        # Dependências
```

O projeto inteiro está em **um único arquivo** (`App.js`) para facilitar a explicação em aula.

Referências

- [Documentação oficial do React Native](https://reactnative.dev) (https://reactnative.dev)
- [Documentação do Expo](https://docs.expo.dev) (https://docs.expo.dev)
- [Expo Go — Google Play](https://play.google.com/store/apps/details?id=host.exp.exponent) (https://play.google.com/store/apps/details?id=host.exp.exponent)

- [Expo Go — App Store](https://apps.apple.com/app/expo-go/id982107779) (https://apps.apple.com/app/expo-go/id982107779)

AULA 03 — COMPONENTES E ESTILOS

Objetivos da Aula

Criar componentes reutilizáveis com Flexbox mobile, aplicar StyleSheet avançado e integrar ícones com @expo/vector-icons e React Native Paper.

O Projeto desta Aula

A partir desta aula, começamos a construir o **minhas-financas** — um app de gerenciamento financeiro pessoal que evoluirá ao longo de todo o módulo.

Ao final da Aula 5, você terá um app real, funcional e publicável.

StyleSheet Avançado

Organização de estilos

Em projetos maiores, manter todos os estilos em um único StyleSheet no final do arquivo já não é suficiente. A melhor prática é criar arquivos de tema e reutilizar variáveis:

```
// theme.js – cores e tamanhos globais
export const cores = {
  primaria: '#2ecc71', // verde – receitas
  perigo: '#e74c3c', // vermelho – despesas
  fundo: '#f5f6fa',
  cartao: '#ffffff',
  texto: '#2c3e50',
  subtexto: '#7f8c8d',
};

export const espacamento = {
  xs: 4,
  sm: 8,
  md: 16,
  lg: 24,
  xl: 32,
};

// Usando o tema no componente:
import { cores, espacamento } from './theme';

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: cores.fundo,
    padding: espacamento.md,
  },
  titulo: {
    fontSize: 22,
    fontWeight: 'bold',
    color: cores.texto,
  },
});
```

```
marginBottom: espacamento.sm,  
  },  
});
```

Composição de estilos com arrays

No React Native você pode passar um **array de estilos** — o último tem prioridade:

```
// Estilo base + variação condicional:  
<View style={[styles.cartao, isDestacado && styles.cartaoDestacado]}>  
  <Text style={[styles.texto, { color: valor > 0 ? '#2ecc71' : '#e74c3c' }]}>  
    R$ {valor}  
  </Text>  
</View>  
  
const styles = StyleSheet.create({  
  cartao: {  
    padding: 16,  
    borderRadius: 12,  
    backgroundColor: '#fff',  
  },  
  cartaoDestacado: {  
    borderWidth: 2,  
    borderColor: '#2ecc71',  
  },  
  texto: {  
    fontSize: 16,  
    fontWeight: '600',  
  },  
});
```

Flexbox Mobile

O React Native usa Flexbox por padrão em **todos** os componentes. A diferença crítica em relação ao CSS web:

Propriedade	CSS Web	React Native
flexDirection padrão	row	column
Unidades	px, %, rem	apenas números (dp)
display: flex	precisa declarar	sempre ativo

As propriedades mais usadas

```
<View style={{  
  flex: 1, // ocupa todo o espaço disponível  
  flexDirection: 'row', // filhos lado a lado (padrão: 'column')  
  justifyContent: 'space-between', // eixo principal: início, fim, centro, espaço  
  alignItems: 'center', // eixo cruzado: centro, início, fim, stretch  
  flexWrap: 'wrap', // quebra linha quando não cabe  
  gap: 8, // espaço entre filhos (RN 0.71+)  
}}>
```

Componentes Reutilizáveis

CartaoSaldo

Exibe o saldo total com cor dinâmica (verde se positivo, vermelho se negativo). Usa `Math.abs()` para sempre mostrar o número positivo e operador ternário para selecionar a cor.

ItemTransacao

Cada linha da lista: ícone de categoria, descrição, data e valor. Usa `flex: 1` na área de texto para empurrar o valor para a borda direita, e `numberOfLines={1}` para truncar descrições longas.

Atividade prática: O código completo de `CartaoSaldo`, `CardsResumo`, `ItemTransacao` e `theme.js` está no conteúdo complementar.

Sombras no Android e iOS

O React Native usa propriedades diferentes para sombra em cada plataforma:

```
const styles = StyleSheet.create({
  card: {
    // Sombra no iOS:
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.06,
    shadowRadius: 4,
    // Sombra no Android:
    elevation: 2,
  },
});
```

`elevation` não existe no iOS; `shadowColor` e companhia não funcionam no Android. Use os dois blocos juntos para ter sombra em ambas as plataformas.

Operadores Úteis no React Native

Nullish Coalescing (??)

```
const nomeIcone = ICONES[categoria] ?? 'ellipsis-horizontal-circle';
// Retorna o lado direito apenas se o esquerdo for null ou undefined
// (diferente do ||, que também ativa para '', 0 e false)
```

Optional Chaining (?.)

```
latitude: localizacao?.latitude ?? null
// Acessa latitude apenas se localizacao não for null/undefined
```

numberOfLines

```
<Text numberOfLines={1}>{descricao}</Text>
// Trunca o texto com "..." após 1 linha – evita quebrar o layout
```

Calculando Totais com filter + reduce

Padrão recorrente no app financeiro:

```
const receitas = transacoes
  .filter(t => t.tipo === 'receita') // filtra apenas receitas
  .reduce((acc, t) => acc + t.valor, 0); // soma os valores (começa em 0)

const despesas = transacoes
  .filter(t => t.tipo === 'despesa')
  .reduce((acc, t) => acc + t.valor, 0);

const saldo = receitas - despesas;
```

O reduce recebe dois parâmetros: a função acumuladora (acc, item) => novoAcc e o **valor inicial** (0). Sem o valor inicial, o reduce usaria o primeiro elemento como ponto de partida, o que quebraria se a lista estivesse vazia.

Ícones com @expo/vector-icons

O Expo já inclui o @expo/vector-icons — não precisa instalar nada extra.

Famílias disponíveis

Biblioteca

Ionicons
MaterialIcons
FontAwesome
Feather

Quando usar

Interface geral — recomendada para iniciantes
Estilo Google/Material Design
Ícones clássicos da web
Ícones modernos e minimalistas

Como usar

```
import { Ionicons } from '@expo/vector-icons';
import { MaterialIcons } from '@expo/vector-icons';

// Básico:
<Ionicons name="home" size={24} color="#333" />

// Com estilo:
<Ionicons name="add-circle" size={32} color="#2ecc71" style={{ marginRight: 8 }} />

// Em um botão:
<TouchableOpacity onPress={handleAdicionar}>
  <Ionicons name="add" size={28} color="#fff" />
</TouchableOpacity>
```

■ **Dica:** Pesquise ícones disponíveis em icons.expo.fyi (https://icons.expo.fyi)

React Native Paper

O React Native Paper é uma biblioteca de componentes no estilo Material Design, pronta para uso em produção.

Instalação

```
npm install react-native-paper
```

Configuração

```
// App.js – envolva tudo com o PaperProvider:
import { PaperProvider } from 'react-native-paper';

export default function App() {
  return (
    <PaperProvider>
      {/* resto do app */}
    </PaperProvider>
  );
}
```

Componentes úteis para o minhas-financas

```
import { Button, Chip, Card, Divider } from 'react-native-paper';

// Botão estilizado:
<Button mode="contained" onPress={handleSalvar} icon="check">
  Salvar Transação
</Button>

// Chips para categorias:
<Chip icon="restaurant" onPress={() => setCategoria('alimentacao')}>
  Alimentação
</Chip>

// Card com elevação:
<Card style={{ margin: 16 }}>
  <Card.Title title="Resumo do Mês" />
  <Card.Content>
    <Text>Receitas: R$ 3.200,00</Text>
  </Card.Content>
</Card>
```

Imagens no React Native

```
import { Image } from 'react-native';

// Imagem local (dentro do projeto):
<Image
  source={require('./assets/logo.png')}
  style={{ width: 120, height: 60 }}
  resizeMode="contain"
/>

// Imagem remota (URL):
<Image
  source={{ uri: 'https://exemplo.com/imagem.jpg' }}
  style={{ width: 200, height: 200, borderRadius: 100 }}
/>
```

resizeMode

cover
contain
stretch
center

Comportamento

Preenche sem distorcer (pode cortar)
Mostra inteiro sem cortar
Estica para preencher (pode distorcer)
Centraliza no tamanho original

Projeto Demo em Sala

O diretório `minhas-financas/` (iniciado nesta aula e evoluído nas próximas) contém a tela principal do app com dados estáticos.

Como rodar

```
cd minhas-financas
npm install
npx expo start
```

O que o demo mostra

Seção

CartaoSaldo
CardsResumo
Lista de transações
Categorias com ícones

Conceito demonstrado

Flexbox, StyleSheet, props, lógica condicional
`flexDirection: 'row', flex: 1` em filhos
Componente `ItemTransacao`, `@expo/vector-icons`, `ScrollView`
Mapeamento de categoria → ícone Ionicons

Estrutura do projeto demo

```
minhas-financas/
├── App.js                # Tela principal montada
├── components/
│   ├── CartaoSaldo.js    # Card de saldo total
│   ├── CardsResumo.js    # Cards de receitas e despesas
│   └── ItemTransacao.js  # Item individual da lista
├── theme.js              # Cores e espaçamentos globais
├── app.json
└── package.json
```

Prepare o Ambiente Local — Obrigatório para a Aula 5

As aulas 2, 3 e 4 podem ser feitas no snack.expo.dev (<https://snack.expo.dev>). A partir da Aula 5, o Snack não é mais suficiente: SQLite, Geolocalização e Mapas exigem ambiente nativo instalado localmente.

Use o tempo desta aula para instalar o que falta:

Item	Status
Node.js 20.x instalado	☐
VS Code instalado	☐
Expo CLI instalado (<code>npm install -g expo-cli</code>)	☐
Android Studio instalado	☐
AVD criado (Pixel 8 / Android 14)	☐
Expo Go instalado no celular	☐

Siga o passo a passo completo em [aula1/README.md](#).

O Android Studio é o passo mais demorado (download ~1 GB + configuração do AVD). Inicie a instalação enquanto acompanha a aula.

Referências

- [Documentação oficial do StyleSheet](https://reactnative.dev/docs/stylesheet) (https://reactnative.dev/docs/stylesheet)
- [Guia de Flexbox — React Native](https://reactnative.dev/docs/flexbox) (https://reactnative.dev/docs/flexbox)
- [Buscador de ícones — icons.expo.fyi](https://icons.expo.fyi) (https://icons.expo.fyi)
- [React Native Paper — componentes](https://callstack.github.io/react-native-paper/docs/components/)

(https://callstack.github.io/react-native-paper/docs/components/)

AULA 04 — NAVEGAÇÃO E ROTAS

Objetivos da Aula

Configurar o React Navigation com Tab e Stack Navigators e passar dados entre telas com `navigation.navigate` e `route.params`.

O Projeto desta Aula

Continuamos o **minhas-financas**. Ao final desta aula, o app terá:

Aba	Tela	O que faz
Dashboard	DashboardScreen	Saldo + lista de transações (Aula 3)
Nova Transação	NovaTransacaoScreen	Formulário para adicionar transação
Relatório	RelatorioScreen	Resumo visual de receitas e despesas

Por que precisamos de Navegação?

Apps reais têm múltiplas telas. Sem navegação, tudo precisaria ficar em um único arquivo — o que rapidamente se torna impossível de manter.

Três padrões de navegação mais comuns no mobile:

Padrão	Exemplo	Componente
Stack (pilha)	Detalhes → Editar → Salvar	Stack.Navigator
Tab (abas)	Dashboard / Perfil / Config	Tab.Navigator
Drawer (gaveta lateral)	Menu hamburguer	Drawer.Navigator

No minhas-financas usaremos **Tab + Stack combinados**: abas na parte inferior e uma tela de formulário empilhada sobre o Dashboard.

Instalação do React Navigation

Passo a passo

```
# 1. Biblioteca principal de navegação
npm install @react-navigation/native
```

```
# 2. Dependências de gestos e animações
npx expo install react-native-screens react-native-safe-area-context

# 3. Navigator de abas inferiores
npm install @react-navigation/bottom-tabs

# 4. Navigator de pilha (para o formulário)
npm install @react-navigation/native-stack
```

Configuração básica

Todo app com React Navigation precisa de um NavigationContainer envolvendo tudo:

```
// App.js
import { NavigationContainer } from '@react-navigation/native';

export default function App() {
  return (
    <NavigationContainer>
      /* navegadores ficam aqui dentro */
    </NavigationContainer>
  );
}
```

Tab Navigator

O Tab Navigator cria a barra de abas na parte inferior da tela.

Estrutura básica

```
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { Ionicons } from '@expo/vector-icons';

const Tab = createBottomTabNavigator();

function TabRoutes() {
  return (
    <Tab.Navigator
      screenOptions={({ route }) => ({
        headerShown: false,
        tabBarActiveTintColor: '#2ecc71',
        tabBarInactiveTintColor: '#95a5a6',
        tabBarStyle: {
          backgroundColor: 'fff',
          borderTopColor: 'eee',
          height: 60,
          paddingBottom: 8,
        },
      })
      tabBarIcon={({ color, size }) => {
        const icon = {
          Dashboard: 'home',
          'Nova Transação': 'add-circle',
          Relatório: 'bar-chart',
        };
        return <Ionicons name={icon[route.name]} size={size} color={color} />;
      }}
    </Tab.Navigator>
  );
}
```

```
    },  
  }}}  
  >  
    <Tab.Screen name="Dashboard" component={DashboardScreen} />  
    <Tab.Screen name="Nova Transação" component={NovaTransacaoScreen} />  
    <Tab.Screen name="Relatório" component={RelatorioScreen} />  
  </Tab.Navigator>  
);  
}
```

screenOptions — personalizando aparência

Propriedade

headerShown: false
tabBarActiveTintColor
tabBarInactiveTintColor
tabBarStyle
tabBarIcon

O que faz

Remove o cabeçalho automático
Cor do ícone/texto da aba ativa
Cor das abas inativas
Estilo da barra inteira
Função que retorna o ícone

Stack Navigator

O Stack Navigator empilha telas. Quando você navega para uma nova tela, ela aparece por cima com uma animação. O botão de voltar remove a tela do topo da pilha.

```
import { createNativeStackNavigator } from '@react-navigation/native-stack';  
  
const Stack = createNativeStackNavigator();  
  
function DashboardStack() {  
  return (  
    <Stack.Navigator>  
      <Stack.Screen  
        name="DashboardMain"  
        component={DashboardScreen}  
        options={{ headerShown: false }}  
      />  
      <Stack.Screen  
        name="DetalhesTransacao"  
        component={DetalhesTransacaoScreen}  
        options={{  
          title: 'Detalhes',  
          headerStyle: { backgroundColor: '#2c3e50' },  
          headerTintColor: '#fff',  
        }}  
      />  
    </Stack.Navigator>  
  );  
}
```

Drawer Navigator

O Drawer Navigator cria um menu deslizante (gaveta) que abre a partir da lateral da tela — o padrão “hamburger” usado em apps como Gmail e Google Maps.

Instalação

```
npm install @react-navigation/drawer  
npx expo install react-native-gesture-handler react-native-reanimated
```

O react-native-reanimated exige que você adicione o plugin ao babel.config.js:

```
// babel.config.js  
module.exports = function(api) {  
  api.cache(true);  
  return {  
    presets: ['babel-preset-expo'],  
    plugins: ['react-native-reanimated/plugin'], // ← adicione esta linha  
  };  
};
```

Reinicie o servidor após alterar o babel.config.js: pressione Ctrl+C e execute `npx expo start` novamente.

Estrutura básica

```
import { createDrawerNavigator } from '@react-navigation/drawer';  
import { Ionicons } from '@expo/vector-icons';  
  
const Drawer = createDrawerNavigator();  
  
function DrawerRoutes() {  
  return (  
    <Drawer.Navigator  
      screenOptions={{  
        drawerActiveTintColor: '#2c3e50',  
        drawerInactiveTintColor: '#95a5a6',  
        drawerStyle: { backgroundColor: '#f5f6fa', width: 260 },  
      }}  
    >  
      <Drawer.Screen  
        name="Dashboard"  
        component={DashboardScreen}  
        options={{  
          drawerIcon: ({ color }) => <Ionicons name="home" size={22} color={color} />,  
        }}  
      />  
      <Drawer.Screen  
        name="Relatório"  
        component={RelatorioScreen}  
        options={{  
          drawerIcon: ({ color }) => <Ionicons name="bar-chart" size={22} color={color} />,  
        }}  
      />  
      <Drawer.Screen  
        name="Configurações"  
        component={ConfiguracoesScreen}  
        options={{  
          drawerIcon: ({ color }) => <Ionicons name="settings" size={22} color={color} />,  
        }}  
      />  
    )  
  );  
}
```

```
    </Drawer.Navigator>  
  );  
}
```

Abrindo e fechando a gaveta programaticamente

```
function DashboardScreen({ navigation }) {  
  return (  
    <View>  
      { /* Abre a gaveta ao tocar no ícone hamburguer: */}  
      <TouchableOpacity onPress={() => navigation.openDrawer()}>  
        <Ionicons name="menu" size={28} color="#fff" />  
      </TouchableOpacity>  
  
      { /* Também é possível fechar: */}  
      <TouchableOpacity onPress={() => navigation.closeDrawer()}>  
        <Text>Fechar menu</Text>  
      </TouchableOpacity>  
  
      { /* Ou alternar: */}  
      <TouchableOpacity onPress={() => navigation.toggleDrawer()}>  
        <Text>Alternar menu</Text>  
      </TouchableOpacity>  
    </View>  
  );  
}
```

Combinando Drawer + Tab

O padrão mais comum em apps complexos é ter um Drawer na raiz e Tabs dentro de uma das telas:

```
Drawer.Navigator  
├─ Tela Principal (Tab.Navigator)  
│   ├── Dashboard  
│   ├── Nova Transação  
│   └── Relatório  
└─ Configurações  
// App.js  
export default function App() {  
  return (  
    <NavigationContainer>  
      <DrawerRoutes />      { /* Drawer envolve tudo */}  
    </NavigationContainer>  
  );  
}  
  
// DrawerRoutes.js  
function DrawerRoutes() {  
  return (  
    <Drawer.Navigator>  
      <Drawer.Screen name="App" component={TabRoutes} /> { /* Tab dentro do Drawer */}  
      <Drawer.Screen name="Configurações" component={ConfiguracoesScreen} />  
    </Drawer.Navigator>  
  );  
}
```

Quando usar Drawer

Muitos itens de menu (5+)
Itens raramente acessados
Apps estilo “painel de controle”

Quando usar Tab

Poucas telas principais (2–4)
Itens acessados com frequência
Apps estilo “redes sociais”

Navegando entre telas

Todo componente de tela recebe automaticamente a prop navigation:

```
function DashboardScreen({ navigation }) {  
  return (  
    <View>  
      <Button  
        title="Ver detalhes da transação"  
        onPress={() => navigation.navigate('DetalhesTransacao', {  
          id: '123',  
          descricao: 'Supermercado',  
          valor: 280.50,  
        })}  
      />  
      <Button title="Voltar" onPress={() => navigation.goBack()} />  
    </View>  
  );  
}
```

Recebendo parâmetros na tela destino

```
function DetalhesTransacaoScreen({ route }) {  
  // route.params contém o que foi passado no navigate()  
  const { id, descricao, valor } = route.params;  
  
  return (  
    <View>  
      <Text>{descricao}</Text>  
      <Text>R$ {valor.toFixed(2)}</Text>  
    </View>  
  );  
}
```

Formulário de Nova Transação

A tela NovaTransacaoScreen precisa de campos de entrada de texto e seleção de categoria.

TextInput — campo de texto

```
import { TextInput } from 'react-native';  
  
function NovaTransacaoScreen() {  
  const [descricao, setDescricao] = useState('');  
  const [valor, setValor] = useState('');  
  
  return (  
    <View style={styles.container}>  
      <Text style={styles.label}>Descrição</Text>
```

```
<TextInput
  style={styles.input}
  value={descricao}
  onChangeText={setDescricao}
  placeholder="Ex: Supermercado"
  maxLength={50}
/>

<Text style={styles.label}>Valor (R$)</Text>
<TextInput
  style={styles.input}
  value={valor}
  onChangeText={setValor}
  placeholder="0,00"
  keyboardType="decimal-pad" // abre teclado numérico
/>
</View>
);
}
```

```
const styles = StyleSheet.create({
  input: {
    borderWidth: 1,
    borderColor: '#ddd',
    borderRadius: 8,
    padding: 12,
    fontSize: 16,
    marginBottom: 16,
    backgroundColor: '#fff',
  },
  label: {
    fontSize: 14,
    fontWeight: '600',
    color: '#555',
    marginBottom: 4,
  },
});
```

Seleção de tipo (Receita / Despesa)

```
function SeletorTipo({ tipo, onChange }) {
  return (
    <View style={styles.seletor}>
      <TouchableOpacity
        style={[styles.botaoTipo, tipo === 'receita' && styles.ativo_receita]}
        onPress={() => onChange('receita')}
      >
        <Ionicons name="arrow-up" size={18} color={tipo === 'receita' ? '#fff' : '#555'} />
        <Text style={[styles.textoTipo, tipo === 'receita' && { color: '#fff' }]}>Receita</Text>
      </TouchableOpacity>

      <TouchableOpacity
        style={[styles.botaoTipo, tipo === 'despesa' && styles.ativo_despesa]}
        onPress={() => onChange('despesa')}
      >
        <Ionicons name="arrow-down" size={18} color={tipo === 'despesa' ? '#fff' : '#555'} />
      </TouchableOpacity>
    </View>
  );
}
```

```
<Text style={[styles.textoTipo, tipo === 'despesa' && { color: '#fff' }]}>Despesa</Text>
</TouchableOpacity>
</View>
);
}
```

Estrutura de Arquivos para Navegação

Projetos com navegação devem organizar as telas em uma pasta separada:

```
minhas-financas/
├─ App.js                      # NavigationContainer + TabRoutes
├─ routes/
│   └─ TabRoutes.js           # Tab.Navigator com as 3 abas
├─ screens/
│   ├── DashboardScreen.js    # Tela principal (Aula 3)
│   ├── NovaTransacaoScreen.js # Formulário de nova transação
│   └─ RelatorioScreen.js     # Resumo de receitas e despesas
├─ components/                # Componentes reutilizáveis (Aula 3)
├─ theme.js
└─ package.json
```

Navegação Condicional

Apps reais mostram telas diferentes dependendo do estado do usuário — por exemplo: mostrar a tela de login se o usuário não estiver autenticado, ou o app principal se estiver.

Conceito

Navegação condicional é controlada por **estado** (não por rotas). O NavigationContainer renderiza navegadores diferentes conforme uma condição:

```
// App.js
import { useState } from 'react';
import { NavigationContainer } from '@react-navigation/native';

export default function App() {
  const [autenticado, setAutenticado] = useState(false);

  return (
    <NavigationContainer>
      {autenticado
        ? <TabRoutes onLogout={() => setAutenticado(false)} /> // app principal
        : <AuthStack onLogin={() => setAutenticado(true)} /> // telas de login
      }
    </NavigationContainer>
  );
}
```

Stack de autenticação

```
import { createNativeStackNavigator } from '@react-navigation/native-stack';
```

```
const Stack = createNativeStackNavigator();

function AuthStack({ onLogin }) {
  return (
    <Stack.Navigator screenOptions={{ headerShown: false }}>
      <Stack.Screen name="Login">
        {(props) => <LoginScreen {...props} onLogin={onLogin} />}
      </Stack.Screen>
      <Stack.Screen name="Cadastro" component={CadastroScreen} />
      <Stack.Screen name="EsqueciSenha" component={EsqueciSenhaScreen} />
    </Stack.Navigator>
  );
}

function LoginScreen({ navigation, onLogin }) {
  return (
    <View style={{ flex: 1, justifyContent: 'center', padding: 24 }}>
      <Text style={{ fontSize: 28, fontWeight: 'bold', marginBottom: 32 }}>Entrar</Text>

      <TouchableOpacity
        style={{ backgroundColor: '#2c3e50', padding: 16, borderRadius: 8, alignItems: 'center' }}
        onPress={onLogin} // chama o callback – troca o navegador no App.js
      >
        <Text style={{ color: 'fff', fontSize: 16, fontWeight: '700' }}>Acessar</Text>
      </TouchableOpacity>

      <TouchableOpacity
        style={{ marginTop: 16, alignItems: 'center' }}
        onPress={() => navigation.navigate('Cadastro')}
      >
        <Text style={{ color: '#2c3e50' }}>Não tem conta? Cadastre-se</Text>
      </TouchableOpacity>
    </View>
  );
}
```

Persistindo o estado de autenticação

Na prática, o estado autenticado precisa ser carregado do AsyncStorage ao abrir o app, para que o usuário não precise fazer login toda vez:

```
import { useState, useEffect } from 'react';
import AsyncStorage from '@react-native-async-storage/async-storage';

export default function App() {
  const [autenticado, setAutenticado] = useState(false);
  const [verificando, setVerificando] = useState(true); // evita flash da tela errada

  useEffect(() => {
    async function verificarSessao() {
      const token = await AsyncStorage.getItem('@app:token');
      setAutenticado(!!token);
      setVerificando(false);
    }
  }, []);
}
```

```
verificarSessao();
}, []);

if (verificando) {
  return <SplashScreen />; // mostra splash enquanto verifica
}

return (
  <NavigationContainer>
    {autenticado
      ? <TabRoutes onLogout={async () => {
        await AsyncStorage.removeItem('@app:token');
        setAutenticado(false);
      }}
      />
      : <AuthStack onLogin={async (token) => {
        await AsyncStorage.setItem('@app:token', token);
        setAutenticado(true);
      }}
      />
    }
  </NavigationContainer>
);
}
```

Por que não usar `navigation.navigate('Login')` para fazer logout? Porque as telas de login e do app principal são navegadores separados — não há como navegar entre eles. A troca é feita alterando o **estado** no `App.js`, e o React re-renderiza o navegador correto automaticamente.

KeyboardAvoidingView — Teclado que Cobre Campos

Em dispositivos iOS (e alguns Android), o teclado virtual pode cobrir campos de formulário. O

`KeyboardAvoidingView` evita isso:

```
import { KeyboardAvoidingView, Platform } from 'react-native';

<KeyboardAvoidingView
  behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
  style={{ flex: 1 }}
>
  <ScrollView keyboardShouldPersistTaps="handled">
    <TextInput placeholder="Descrição" />
    <TextInput placeholder="Valor" keyboardType="decimal-pad" />
  </ScrollView>
</KeyboardAvoidingView>
```

behavior
'padding'
'height'
'position'

Efeito
Adiciona espaço abaixo do campo
Reduz a altura do container
Reposiciona o container

Recomendado para
iOS
Android
Casos especiais

A prop `keyboardShouldPersistTaps="handled"` no `ScrollView` garante que tocar fora do teclado não desmonte os campos antes do `onPress` ser processado.

Recebendo Dados via route.params com useEffect

Ao usar `navigation.navigate('Dashboard', { novaTransacao })` para passar dados de volta, a tela destino precisa de um `useEffect` para reagir à mudança:

```
function DashboardScreen({ navigation, route }) {
  const [transacoes, setTransacoes] = useState(INICIAIS);

  // Reage sempre que route.params?.novaTransacao mudar
  useEffect(() => {
    if (route.params?.novaTransacao) {
      setTransacoes(prev => [route.params.novaTransacao, ...prev]);
    }
  }, [route.params?.novaTransacao]);
  // ...
}
```

Por que useEffect e não verificar direto no render? Porque os parâmetros podem chegar depois da primeira renderização. O `useEffect` com a dependência correta garante que o código execute apenas quando o valor realmente mudar.

Passando Callback entre Telas

Na Aula 5 usaremos Context API, mas por enquanto podemos passar uma função de callback pelo `navigate`:

```
// DashboardScreen – passa função para receber nova transação
function DashboardScreen({ navigation }) {
  const [transacoes, setTransacoes] = useState(TRANSACOES_INICIAIS);

  const adicionarTransacao = (novaTransacao) => {
    setTransacoes(prev => [novaTransacao, ...prev]);
  };

  return (
    <View>
      <Button
        title="+ Nova"
        onPress={() => navigation.navigate('Nova Transação', { onSalvar: adicionarTransacao })}
      />
      { /* lista de transacoes... */ }
    </View>
  );
}

// NovaTransacaoScreen – chama o callback ao salvar
function NovaTransacaoScreen({ route, navigation }) {
  const { onSalvar } = route.params ?? {};

  const salvar = () => {
    const novaTransacao = { id: Date.now().toString(), descricao, valor: parseFloat(valor), tipo, categoria };
    onSalvar?.(novaTransacao); // chama o callback se existir
    navigation.goBack(); // volta para o Dashboard
  };
}
```

```
};  
  
// ...  
}
```

Projeto Demo em Sala

Atividade prática: O código completo de todas as telas (DashboardScreen, NovaTransacaoScreen, RelatorioScreen, DetalheTransacaoScreen, BoasVindasScreen) e dos navegadores (TabRoutes, DashboardStack) está no conteúdo complementar.

Como rodar

```
cd minhas-financas  
npm install  
npx expo start
```

O que o demo mostra

Aba/Tela

Barra inferior com ícones
Dashboard → Nova Transação
Formulário com campos
Botão “Salvar” → volta
Tela de Relatório

Conceito demonstrado

Tab Navigator + tabBarIcon
navigation.navigate() com params
TextInput, useState, validação básica
navigation.goBack() + callback
Cálculo de totais, exibição de dados

Estrutura do projeto demo

```
minhas-financas/  
├── App.js  
├── routes/  
│   └── TabRoutes.js  
├── screens/  
│   ├── DashboardScreen.js  
│   ├── NovaTransacaoScreen.js  
│   └── RelatorioScreen.js  
├── components/  
│   ├── CartaoSaldo.js  
│   ├── CardsResumo.js  
│   └── ItemTransacao.js  
└── theme.js
```

Referências

- [React Navigation — documentação oficial](https://reactnavigation.org/docs/getting-started) (https://reactnavigation.org/docs/getting-started)
- [Bottom Tab Navigator](https://reactnavigation.org/docs/bottom-tab-navigator) (https://reactnavigation.org/docs/bottom-tab-navigator)
- [Native Stack Navigator](https://reactnavigation.org/docs/native-stack-navigator) (https://reactnavigation.org/docs/native-stack-navigator)
- [Drawer Navigator](https://reactnavigation.org/docs/drawer-navigator) (https://reactnavigation.org/docs/drawer-navigator)
- [Passagem de parâmetros](https://reactnavigation.org/docs/params) (https://reactnavigation.org/docs/params)

- [Autenticação com React Navigation](https://reactnavigation.org/docs/auth-flow) (https://reactnavigation.org/docs/auth-flow)

AULA 05 — APIS E ARMAZENAMENTO

Objetivos da Aula

Persistir dados com AsyncStorage, implementar CRUD completo, compartilhar estado global via Context API e gerenciar estados de carregamento e erro.

Pré-Requisitos — Configure Antes de Começar

Ambiente base (deve estar instalado)

Item	Verificar com
Node.js 20.x	<code>node -v</code>
Expo CLI	<code>expo --version</code>
Android Studio + AVD rodando	<code>adb devices</code> — deve listar o emulador
Projeto minhas-financas das aulas anteriores	<code>npx expo start</code> + tecla a abre no emulador

Se o emulador não estiver configurado, siga o guia em [aula1/README.md](#) antes de continuar.

Pacotes novos — instalar no início da aula

Dentro da pasta minhas-financas, execute:

```
# Cliente HTTP (alternativa ao fetch)
npm install axios

# Armazenamento chave-valor persistente
npx expo install @react-native-async-storage/async-storage
```

Use `npx expo install` (não `npm install`) para os pacotes Expo: ele garante a versão compatível com o seu SDK automaticamente.

Ferramenta de teste de API

Instale uma das opções abaixo para testar endpoints antes de integrar no app:

Ferramenta	Download	Ideal para
Postman	postman.com/downloads (https://www.postman.com/downloads)	Explorar APIs com interface completa
Insomnia	insomnia.rest/download (https://insomnia.rest/download)	Interface mais leve e rápida
Thunder Client	Extensão do VS Code	Quem prefere tudo dentro do editor

O Projeto desta Aula

Esta é a aula mais importante do módulo do ponto de vista prático: o **minhas-financas** passa de uma demonstração estática para um **app completamente funcional**.

O que muda nesta aula

Transações são **salvas no dispositivo** — persistem ao fechar e reabrir o app
O formulário da Aula 4 **realmente adiciona** transações à lista
É possível **excluir** transações
O saldo é **calculado em tempo real** a partir dos dados salvos
Um **ActivityIndicator** aparece enquanto os dados carregam

AsyncStorage — Armazenamento Local

O AsyncStorage é o equivalente mobile do localStorage do navegador: um banco de chave-valor simples que persiste os dados no dispositivo, mesmo após o app ser fechado.

Instalação

```
npx expo install @react-native-async-storage/async-storage
```

Como funciona

```
AsyncStorage
├─ chave: '@minhasfinancas:transacoes' → valor: ' [{"id": "1", ...}, {"id": "2", ...} ]'
├─ chave: '@minhasfinancas:config'     → valor: '{"moeda": "BRL"}'
└─ ...
```

Os valores **sempre são strings** — objetos e arrays precisam ser convertidos com `JSON.stringify()` ao salvar e `JSON.parse()` ao ler.

API básica

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// SALVAR:
await AsyncStorage.setItem('chave', JSON.stringify(dados));

// LER:
const json = await AsyncStorage.getItem('chave');
const dados = json ? JSON.parse(json) : null;

// REMOVER:
await AsyncStorage.removeItem('chave');

// LISTAR TODAS AS CHAVES:
const chaves = await AsyncStorage.getAllKeys();
```

■ Todas as operações são **assíncronas** — sempre use `await` ou `.then()`.

Context API — Estado Global

Nas aulas anteriores, as transações eram um estado local do DashboardScreen. O problema: quando a NovaTransacaoScreen adiciona uma transação, precisamos passar o dado “de volta” pelo `navigation.navigate` — funciona, mas é frágil.

A solução é o **Context API**: um estado global que qualquer tela pode ler ou modificar, sem precisar de props ou parâmetros de navegação.

Analogia

Imagine um quadro de avisos no centro do escritório. Qualquer funcionário (componente) pode ler o quadro ou afixar um papel. Ninguém precisa entregar recados pessoalmente de um para o outro.

Criando o contexto de transações

O TransacoesContext encapsula toda a lógica de persistência:

Função

carregarTransacoes()
adicionarTransacao(t)
removerTransacao(id)
receitas, despesas, saldo

O que faz

Lê as transações do AsyncStorage ao abrir o app
Adiciona ao estado + salva no AsyncStorage
Remove do estado + salva no AsyncStorage
Calculados automaticamente com `reduce()`

Qualquer tela pode acessar o estado com o hook customizado:

```
const { transacoes, saldo, receitas, despesas, carregando, removerTransacao } = useTransacoes();
```

Atividade prática: O código completo de TransacoesContext, DashboardScreen, NovaTransacaoScreen e o hook de cotações está no conteúdo complementar.

Estados de Carregamento e Erro

Boas UX's sempre indicam ao usuário o que está acontecendo. Três estados importantes:

Loading (carregando)

```
import { ActivityIndicator } from 'react-native';

// Enquanto carrega, exibe o spinner:
if (carregando) {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <ActivityIndicator size="large" color="#2c3e50" />
      <Text style={{ marginTop: 12, color: '#555' }}>Carregando transações...</Text>
    </View>
  );
}
```

Empty state (sem dados)

```
// Quando não há transações:
if (transacoes.length === 0) {
  return (
    <View style={styles.vazio}>
      <Ionicons name="wallet-outline" size={64} color="#bdc3c7" />
      <Text style={styles.textoVazio}>Nenhuma transação ainda</Text>
      <Text style={styles.subtextoVazio}>Toque em "Nova Transação" para começar</Text>
    </View>
  );
}
```

Tratamento de erro

```
async function carregarTransacoes() {
  try {
    setCarregando(true);
    const json = await AsyncStorage.getItem(CHAVE);
    if (json) setTransacoes(JSON.parse(json));
  } catch (erro) {
    Alert.alert(
```

```
    'Erro',  
    'Não foi possível carregar as transações. Tente novamente.',  
    [{ text: 'Tentar novamente', onPress: carregarTransacoes }]  
  );  
} finally {  
  setCarregando(false); // sempre executa, mesmo com erro  
}  
}
```

Excluir Transação com Confirmação

```
import { Alert } from 'react-native';  
  
function confirmarExclusao(id, descricao) {  
  Alert.alert(  
    'Excluir transação',  
    `Deseja excluir "${descricao}"?`,  
    [  
      { text: 'Cancelar', style: 'cancel' },  
      {  
        text: 'Excluir',  
        style: 'destructive',  
        onPress: () => removerTransacao(id),  
      },  
    ],  
  );  
}  
  
// No ItemTransacao, adicione onLongPress:  
<TouchableOpacity  
  onPress={onPress}  
  onLongPress={() => confirmarExclusao(transacao.id, transacao.descricao)}  
  style={styles.container}  
>
```

Custom Hooks — Reutilizando Lógica

Um **custom hook** é uma função que começa com `use` e encapsula lógica reutilizável com hooks do React. Ele permite separar lógica de negócio do componente visual:

O custom hook encapsula os três estados da requisição (cotacoes, carregando, erro) e expõe a função atualizar para o botão de refresh. O componente fica responsável apenas pela UI:

```
const { cotacoes, carregando, erro, atualizar } = useCotacoes();
```

Por que criar um custom hook em vez de colocar no componente?

- O componente fica responsável apenas pela UI — sem lógica de fetch
- O hook pode ser reutilizado em qualquer outra tela
- Fica fácil de testar e depurar em isolamento

O código completo de `useCotacoes` e `CartaoCotacoes` está no conteúdo complementar.

Consumindo uma API REST

Além do armazenamento local, apps reais consomem APIs externas. No `minhas-financas`, vamos buscar a cotação do dólar para exibir no dashboard como informação adicional.

fetch vs Axios

Existem duas formas principais de fazer requisições HTTP no React Native:

	fetch	Axios
Instalação	Nativo (não precisa instalar)	<code>npm install axios</code>
Verificação de erro HTTP	Manual (<code>if (!res.ok)</code>)	Automática (lança erro em <code>4xx/5xx</code>)
Timeout	Manual	Nativo (<code>timeout: 5000</code>)
Interceptors	Não	Sim (útil para tokens JWT)
Transformação de JSON	Manual (<code>.json()</code>)	Automática (<code>res.data</code>)

Com fetch (nativo):

```
const resposta = await fetch('https://api.exemplo.com/dados');
if (!resposta.ok) throw new Error(`Erro: ${resposta.status}`);
const dados = await resposta.json();
```

Com Axios (biblioteca):

```
import axios from 'axios';

// Instância configurada (boas práticas em projetos maiores):
const api = axios.create({
  baseURL: 'https://api.exemplo.com',
  timeout: 5000,
  headers: { 'Content-Type': 'application/json' },
});

// Requisição – sem precisar converter .json() manualmente:
const { data } = await api.get('/dados');

// POST com body:
const { data: nova } = await api.post('/transacoes', {
  descricao: 'Supermercado',
  valor: 150,
});
```

Para projetos simples como o `minhas-financas`, o `fetch` é suficiente. Use `Axios` quando o projeto crescer ou precisar de interceptors para autenticação.

Usando a API pública do Banco Central

```
// Usando a API pública do Banco Central (gratuita, sem autenticação):
const URL_DOLAR = 'https://economia.awesomeapi.com.br/json/last/USD-BRL';

async function buscarCotacaoDolar() {
  try {
    const resposta = await fetch(URL_DOLAR);
```

```
if (!resposta.ok) {  
  throw new Error(`Erro HTTP: ${resposta.status}`);  
}  
  
const dados = await resposta.json();  
const cotacao = parseFloat(dados.USDBRL.bid);  
setCotacaoDolar(cotacao);  
} catch (erro) {  
  console.warn('Não foi possível buscar cotação:', erro.message);  
  // Não exibe erro para o usuário – é uma informação opcional  
}  
}  
  
useEffect(() => {  
  buscarCotacaoDolar();  
}, []);
```

Projeto Demo em Sala

Como rodar

```
cd minhas-financas  
npm expo start
```

O que o demo mostra

Funcionalidade

Dados persistem ao fechar e abrir
Spinner de carregamento
Formulário salva de verdade
Toque longo para excluir
Tela vazia motivacional
Cotação do dólar

Conceito demonstrado

AsyncStorage setItem / getItem
ActivityIndicator + estado carregando
adicionarTransacao() via Context
onLongPress + Alert.alert confirmação
Empty state com ícone
fetch com API pública

Estrutura do projeto demo

```
minhas-financas/  
├── App.js  
├── context/  
│   └── TransacoesContext.js    # Estado global + AsyncStorage  
├── routes/  
│   └── TabRoutes.js  
├── screens/  
│   ├── DashboardScreen.js      # usa useTransacoes()  
│   ├── NovaTransacaoScreen.js  # usa adicionarTransacao()  
│   └── RelatorioScreen.js       # usa receitas, despesas, saldo  
├── components/  
│   ├── CartaoSaldo.js  
│   ├── CardsResumo.js  
│   └── ItemTransacao.js  
└── theme.js
```

Referências

- [AsyncStorage — documentação](https://react-native-async-storage.github.io/async-storage/) (https://react-native-async-storage.github.io/async-storage/)
- [Axios — documentação](https://axios-http.com/ptbr/docs/intro) (https://axios-http.com/ptbr/docs/intro)
- [Context API — documentação React](https://react.dev/reference/react/createContext) (https://react.dev/reference/react/createContext)
- [ActivityIndicator — React Native](https://reactnative.dev/docs/activityindicator) (https://reactnative.dev/docs/activityindicator)
- [API de cotações \(gratuita\)](https://docs.awesomeapi.com.br/api-de-moedas) (https://docs.awesomeapi.com.br/api-de-moedas)

AULA 06 — ARMAZENAMENTO COM SQLITE

Objetivos da Aula

Comparar AsyncStorage e SQLite, criar e operar um banco local com expo-sqlite e escrever consultas com WHERE, ORDER BY e SUM.

Pré-Requisitos — Configure Antes de Começar

Ambiente base (deve estar instalado)

Item	Verificar com
Node.js 20.x	node -v
Expo CLI	expo --version
Android Studio + AVD rodando	adb devices
Projeto minhas-financas das aulas 3-5	npx expo start + tecla a

Pacote novo — instalar no início da aula

```
npx expo install expo-sqlite
```

Conteúdo Teórico

O que é SQLite?

SQLite é um banco de dados relacional que roda localmente no dispositivo — sem servidor, sem conexão de rede. É o banco de dados mais utilizado no mundo, presente em todos os iPhones, Androids, browsers e sistemas operacionais.

Por que usar SQLite em vez de AsyncStorage?

O AsyncStorage é ótimo para armazenar preferências e listas pequenas, mas tem limitações quando os dados crescem ou precisam de filtros. O SQLite resolve isso:

AsyncStorage:

salvar → JSON.stringify(lista inteira) → sobrescreve tudo
ler → JSON.parse(tudo) → filtra no JavaScript

SQLite:

salvar → INSERT INTO transacoes VALUES (...)
ler → SELECT * FROM transacoes WHERE tipo = 'despesa'

Conceitos SQL usados nesta aula

Comando	Exemplo	O que faz
CREATE TABLE	CREATE TABLE IF NOT EXISTS transacoes (...)	Cria a tabela
INSERT INTO	INSERT INTO transacoes VALUES (?, ...)	Adiciona um registro
SELECT	SELECT * FROM transacoes	Busca registros
DELETE	DELETE FROM transacoes WHERE id = ?	Remove um registro
WHERE	WHERE tipo = 'despesa'	Filtra resultados
ORDER BY	ORDER BY rowid DESC	Ordena resultados
SUM	SELECT SUM(valor) FROM transacoes	Soma valores

API Síncrona do expo-sqlite

O expo-sqlite moderno oferece métodos síncronos que simplificam o código:

```
db.execSync('CREATE TABLE ...') // executa sem retorno
db.runSync('INSERT INTO ...', params) // executa com parâmetros
db.getAllSync('SELECT * FROM ...') // retorna array de objetos
db.getFirstSync('SELECT ...') // retorna primeiro resultado
```

Por que síncrono se operações de I/O costumam ser async? O expo-sqlite executa o banco em uma thread separada internamente. A API síncrona é uma abstração que não trava a UI — você ganha a simplicidade de código linear sem async/await.

Comparação prática: AsyncStorage vs SQLite

AsyncStorage — como ele trabalha:

```
salvar → JSON.stringify(lista inteira) → sobrescreve tudo no disco
ler → JSON.parse(tudo) → filtra no JavaScript
```

SQLite — como ele trabalha:

```
salvar → INSERT INTO transacoes VALUES (?) ← só o novo registro
ler → SELECT * FROM transacoes WHERE tipo = 'despesa' ← filtrado no banco
```

A diferença cresce com o volume: para 10 transações AsyncStorage é ótimo; para 10.000 transações SQLite é obrigatório.

Consultas avançadas com SQL

Uma das grandes vantagens do SQLite é filtrar diretamente no banco sem carregar tudo para o JavaScript — WHERE, SUM e BETWEEN executam no banco antes de qualquer código JavaScript.

Atividade prática (bônus): As funções buscarPorCategoria, totalPorTipo e buscarPorPeriodo estão no conteúdo complementar.

Migração de AsyncStorage para SQLite

Ao trocar de AsyncStorage para SQLite em produção, os dados existentes precisam ser migrados. A estratégia recomendada:

```
useEffect(() => {
```

```
inicializarBanco();

// Na primeira abertura após atualização, migra dados antigos:
async function migrarSeNecessario() {
  const jaFezMigracao = await AsyncStorage.getItem('@app:migrado');
  if (jaFezMigracao) return;

  const json = await AsyncStorage.getItem('@minhasfinancas:transacoes');
  if (json) {
    const transacoes = JSON.parse(json);
    transacoes.forEach(t => inserirTransacao(t));
  }
  await AsyncStorage.setItem('@app:migrado', 'sim');
}

migrarSeNecessario();
carregarTransacoes();
}, []);
```

Estrutura de arquivos ao final da aula

```
minhas-financas/
├── database/
│   └── db.js           ← NOVO: helper do banco SQLite
├── context/
│   └── TransacoesContext.js ← ATUALIZADO: usa SQLite
└── ...
```

Projeto Demo em Sala

Como rodar

```
cd minhas-financas
npx expo start
```

Atenção: expo-sqlite não funciona na versão web (localhost:8082). Sempre teste no emulador Android (tecla a) ou no celular com Expo Go.

O que o demo mostra

Funcionalidade

Banco criado automaticamente
Transações persistem ao fechar
Exclusão por id
Funções avançadas (bônus)
API síncrona sem async/await

Conceito demonstrado

CREATE TABLE IF NOT EXISTS
INSERT INTO + SELECT *
DELETE FROM WHERE id = ?
WHERE, SUM, BETWEEN
execSync, runSync, getAllSync

Estrutura do projeto demo

```
minhas-financas/
├── database/
│   └── db.js           # helper com todas as funções SQL
├── context/
│   └── TransacoesContext.js # usa SQLite (não mais AsyncStorage)
```

└ ...

Referências

- [Documentação expo-sqlite](https://docs.expo.dev/versions/latest/sdk/sqlite/) (https://docs.expo.dev/versions/latest/sdk/sqlite/)
- [Tutorial SQLite — W3Schools](https://www.w3schools.com/sql/) (https://www.w3schools.com/sql/)
- [SQLite Browser](https://sqlitebrowser.org/) (https://sqlitebrowser.org/) — visualizar o banco no computador

AULA 07 — GEOLOCALIZAÇÃO E MAPAS

Objetivos da Aula

Solicitar permissões de localização, capturar coordenadas GPS com expo-location, exibir mapa com marcadores via react-native-maps e associar localização às transações no SQLite.

Pré-Requisitos — Configure Antes de Começar

Ambiente base (deve estar instalado)

Item	Verificar com
Node.js 20.x	node -v
Expo CLI	expo --version
Android Studio + AVD rodando	adb devices
Projeto minhas-financas das aulas 3–6	npx expo start + tecla a

Pacotes novos — instalar no início da aula

```
npx expo install expo-location react-native-maps
```

Conteúdo Teórico

Como funciona a geolocalização em apps mobile

O dispositivo determina sua posição combinando três fontes:

Fonte	Precisão	Consumo de bateria
GPS (satélite)	Alta (~3m)	Alto
Wi-Fi	Média (~15m)	Baixo
Torres de celular	Baixa (~100m)	Muito baixo

O expo-location com Accuracy.Balanced escolhe automaticamente a melhor combinação.

Permissões de localização

Apps mobile precisam de permissão explícita do usuário para acessar o GPS. Existem dois tipos:

- **Foreground** (requestForegroundPermissionsAsync) — somente enquanto o app está em uso
- **Background** (requestBackgroundPermissionsAsync) — mesmo com o app fechado

Nesta aula usamos apenas **foreground** — é o mais comum e aceito pelas lojas.

O que são coordenadas geográficas?

latitude → posição norte-sul (-90 a +90)
longitude → posição leste-oeste (-180 a +180)

São Paulo: latitude -23.5505, longitude -46.6333
Brasília: latitude -15.7801, longitude -47.9292

Níveis de precisão do expo-location

Constante	Precisão	Consumo	Quando usar
Accuracy.Lowest	~3km	Mínimo	Não recomendado
Accuracy.Low	~1km	Muito baixo	Cidade aproximada
Accuracy.Balanced	~100m	Baixo	Recomendado — uso geral
Accuracy.High	~10m	Médio	Navegação turn-by-turn
Accuracy.Highest	~3m	Alto	Topografia, corridas
Accuracy.BestForNavigation	~1m	Muito alto	GPS especializado

Para marcar onde uma transação foi feita, Balanced é ideal — precisão suficiente sem drenar a bateria.

react-native-maps

O componente MapView renderiza um mapa interativo usando: - **Google Maps** no Android - **Apple**

Maps no iOS

Principais componentes:

```
<MapView initialRegion={regiao}>
  <Marker
    coordinate={{ latitude, longitude }}
    pinColor="#e74c3c" // vermelho para despesa
    onPress={() => console.log('pin tocado')}
  >
  { /* Callout – balão exibido ao tocar no pin */ }
  <Callout tooltip>
    <View style={{ padding: 8, backgroundColor: '#fff', borderRadius: 8 }}>
      <Text style={{ fontWeight: 'bold' }}>Supermercado</Text>
      <Text style={{ color: '#e74c3c' }}>- R$ 150,00</Text>
      <Text style={{ color: '#999', fontSize: 12 }}>07/04/2026</Text>
    </View>
  </Callout>
</Marker>
</MapView>
```

- `initialRegion`: define o centro e o zoom do mapa ao abrir (`latitudeDelta` e `longitudeDelta` controlam o zoom — valores menores = mais próximo)
- `pinColor`: cor do pin (use as cores do tema para receitas/despesas)
- `Callout tooltip`: remove o estilo padrão do balão e permite total customização
- `showsUserLocation={true}`: exibe o ponto azul indicando a posição atual do usuário

Estrutura de arquivos ao final da aula

```
minhas-financas/
├── hooks/
│   └── useLocalizacao.js      ← NOVO: captura GPS
├── screens/
│   └── MapaScreen.js        ← NOVO: tela do mapa
├── routes/
│   └── TabRoutes.js          ← ATUALIZADO: aba Mapa
├── database/
│   └── db.js                  ← ATUALIZADO: colunas lat/long
└── screens/
    └── NovaTransacaoScreen.js ← ATUALIZADO: botão de localização
```

Projeto Demo em Sala

Atividade prática: O código completo de `useLocalizacao`, `MapaScreen`, `NovaTransacaoScreen` (com localização) e a atualização do `db.js` está no conteúdo complementar.

Como rodar

```
cd minhas-financas
npx expo start
```

Pressione a para abrir no emulador Android. Para simular localização no emulador: no Android Studio → Extended Controls (...) → Location → defina as coordenadas e clique em “Send”.

O que o demo mostra

Funcionalidade

Botão “Capturar localização”

Tela de Mapa com pins

Cor do pin por tipo

Toque no pin exibe detalhes

Tela vazia sem localização

Localização salva no SQLite

Conceito demonstrado

`requestForegroundPermissionsAsync` + `getCurrentPositionAsync`

`MapView`, `Marker`, `Callout`

`pinColor` condicional (verde = receita, vermelho = despesa)

`Callout tooltip` customizado

Empty state com ícone

Colunas `latitude` e `longitude` opcionais

Estrutura do projeto demo

```
minhas-financas/
├── hooks/
│   └── useLocalizacao.js      # encapsula permissão + GPS
├── screens/
│   └── MapaScreen.js        # tela com MapView e Markers
├── routes/
│   └── TabRoutes.js          # aba Mapa adicionada
├── database/
│   └── db.js                  # colunas latitude e longitude
```

Observações sobre produção

Chave de API do Google Maps (Android)

No Expo Go e builds de desenvolvimento, o mapa funciona sem chave. Para builds de produção (APK/AAB), é necessário:

1. Criar uma chave em console.cloud.google.com (<https://console.cloud.google.com>)
2. Ativar “Maps SDK for Android”
3. Adicionar no `app.json`:

```
"android": {  
  "config": {  
    "googleMaps": { "apiKey": "SUA_CHAVE_AQUI" }  
  }  
}
```

Configurar localização no emulador

No Android Studio: Extended Controls (...) → Location → defina as coordenadas e clique em “Send”.

Referências

- [Documentação expo-location](https://docs.expo.dev/versions/latest/sdk/location/) (<https://docs.expo.dev/versions/latest/sdk/location/>)
- [Documentação react-native-maps](https://github.com/react-native-maps/react-native-maps) (<https://github.com/react-native-maps/react-native-maps>)
- [Google Maps Platform](https://console.cloud.google.com) (<https://console.cloud.google.com>) — chaves de API

AULA 08 — PUBLICAÇÃO E DEPLOY

Objetivos da Aula

Configurar o `app.json` para produção, criar assets (ícone e splash), compilar com EAS Build e publicar na Google Play e App Store com OTA updates.

O `app.json` — Identidade do App

O `app.json` é o documento de identidade do seu aplicativo. Ele controla tudo que não faz parte do código JavaScript: nome, versão, ícone, permissões, identificadores de plataforma.

Campos essenciais

Campo	O que define
<code>name</code>	Nome exibido embaixo do ícone na tela do celular
<code>slug</code>	Identificador URL-amigável (sem espaços, sem acentos)
<code>version</code>	Versão exibida na loja (“1.0.0”)
<code>android.package</code>	ID único do app no Android — usa notação de domínio invertido (<code>com.seunome.appname</code>)
<code>android.versionCode</code>	Número inteiro que só sobe a cada build enviado à loja (1, 2, 3...)
<code>ios.bundleIdentifier</code>	ID único no iOS — mesmo padrão do Android, nunca pode ser alterado após publicação
<code>ios.buildNumber</code>	String que sobe a cada build iOS (“1”, “2”...)

Por que domínio invertido? O padrão `com.empresa.app` garante unicidade global. Qualquer pessoa no mundo pode registrar o mesmo nome de app, mas o package ID precisa ser único em toda a loja.

Versionamento semântico

A versão visível ao usuário segue o padrão MAJOR.MINOR.PATCH:

MAJOR – mudanças incompatíveis (redesign completo, nova arquitetura)
MINOR – novas funcionalidades sem quebrar nada (nova tela, novo relatório)
PATCH – correções de bugs (crash resolvido, cálculo errado corrigido)

O `versionCode` (Android) e o `buildNumber` (iOS) são números internos que as lojas usam para identificar qual build é mais recente. Eles **sempre sobem** — nunca podem ser iguais ou menores que o build anterior.

Ícone e Splash Screen

As lojas exigem assets em dimensões específicas. O Expo cuida de gerar os tamanhos menores a partir das imagens originais, então você só precisa fornecer as versões grandes.

Asset	Dimensão	Formato	Uso
<code>icon.png</code>	1024×1024 px	PNG sem transparência	iOS e Android
<code>adaptive-icon.png</code>	1024×1024 px	PNG com transparência	Android (ícones adaptativos)
<code>splash.png</code>	1284×2778 px	PNG	Tela de carregamento

Por que Android tem dois ícones? O sistema de ícones adaptativos do Android (introduzido no Android 8) separa o fundo do ícone (definido pelo sistema/fabricante) do elemento frontal (o seu desenho). Isso permite que o launcher aplique formas diferentes (círculo, quadrado arredondado) sem distorcer o ícone.

Splash screen: aparece por menos de 2 segundos enquanto o app carrega. Mantenha simples — logo e fundo, sem texto desnecessário.

EAS Build — Compilação na Nuvem

O Expo Application Services (EAS) compila o app nos servidores da Expo, eliminando a necessidade de ter Android Studio configurado (para Android) ou um Mac com Xcode (para iOS).

Por que EAS em vez de expo build?

	expo build (antigo)	EAS Build (atual)
Status	Depreciado desde 2023	Recomendado oficialmente
Customização	Limitada	Total (bare workflow)
Velocidade	Lenta	Significativamente mais rápida

Os três perfis de build

O arquivo `eas.json` define os perfis de build. Cada perfil serve a um momento diferente do ciclo de vida:

Perfil	Momento de uso	Formato gerado
<code>development</code>	Desenvolvimento ativo — inclui ferramentas de debug	APK com Expo Dev Client

Perfil	Momento de uso	Formato gerado
preview	Distribuição para testadores sem passar pela loja	APK instalável diretamente
production	Publicação na loja	AAB (Google Play) / IPA (App Store)

APK vs AAB: O APK é o formato de instalação direta — funciona em qualquer Android. O AAB (Android App Bundle) é enviado à Google Play, que então gera APKs otimizados para cada dispositivo (menor download para o usuário final).

Keystore — Assinatura do App

Todo app Android precisa ser assinado com uma chave criptográfica antes de ser distribuído. Essa chave é armazenada no **Keystore**.

- O EAS gera e armazena o Keystore automaticamente na primeira vez
- Sem o Keystore original, é impossível enviar atualizações para o mesmo app na loja
- **Guarde o Keystore em local seguro** — perder significa criar um app novo do zero

Google Play — Fluxo de Publicação

A Google Play organiza a distribuição em faixas progressivas:

Teste Interno → Teste Fechado → Teste Aberto → Produção

Faixa	Quem pode instalar	Revisão da Google	Limite
Teste interno	Lista de e-mails manual	Não	100 pessoas
Teste fechado (Alpha)	Lista de e-mails / grupo	Não	2.000 pessoas
Teste aberto (Beta)	Qualquer um com o link	Sim	Ilimitado
Produção	Todos na loja	Sim	Ilimitado

Por que começar pelo Teste Interno? Porque é a única faixa sem revisão e sem limite de tempo — o app fica disponível em minutos. É ideal para validar que o build funciona antes de se comprometer com o processo de revisão.

Taxa de desenvolvedor: US\$ 25, paga uma vez, válida para sempre. A aprovação da conta pode levar até 48 horas — crie com antecedência.

App Store — Fluxo de Publicação

A Apple exige revisão humana para qualquer distribuição pública. O processo segue este caminho:

Build (EAS) → TestFlight (teste interno) → Revisão da Apple → App Store

Etapa	Descrição
EAS Build	Compila o .ipa nos servidores (não precisa de Mac)
TestFlight	Programa oficial de testes da Apple — distribui para até 10.000 usuários sem revisão pública
Revisão	Equipe da Apple analisa o app (1 a 3 dias úteis em média)

Etapa

App Store

Descrição

Publicação pública

Diferença de custo: US\$ 99/ano (contra US\$ 25 único da Google). A assinatura anual é obrigatória para manter o app na loja.

Comparativo

	Google Play	App Store
Conta	US\$ 25 (taxa única)	US\$ 99/ano
Precisa de Mac?	Não	Não (com EAS Build)
Tempo de revisão	Horas a 1 dia	1 a 3 dias úteis
Teste sem loja	APK direto	TestFlight
Formato	.aab / .apk	.ipa

OTA Updates — Atualizações sem Build

Uma das vantagens do Expo é poder atualizar o JavaScript do app **sem um novo build** e sem passar pela revisão da loja. Isso é chamado de OTA (Over-The-Air).

Quando usar OTA vs novo build

Tipo de mudança

Correção de bug em lógica JS
Nova tela ou componente
Mudança de cores e textos
Nova permissão no sistema
Instalação de biblioteca nativa
Mudança de ícone ou splash

OTA é suficiente?

✓ sim
✓ sim
✓ sim
✗ novo build
✗ novo build
✗ novo build

Por que essa limitação? O OTA atualiza apenas o bundle JavaScript. Qualquer coisa que envolva código nativo (Java/Kotlin no Android, Swift/Obj-C no iOS) exige recompilação — e portanto novo build.

Atividade prática: O passo a passo completo — configuração do app.json, criação do ícone, build com EAS e publicação na Google Play — está no conteúdo complementar.

Checklist de Produção

Antes de publicar, verifique cada item:

Assets

- ☐ icon.png 1024×1024 sem transparência
- ☐ adaptive-icon.png 1024×1024 com transparência
- ☐ splash.png 1284×2778

app.json

- ☐ name definido corretamente
- ☐ version definida (“1.0.0”)
- ☐ android.package único (ex: com.seunome.minhasfinancas)
- ☐ android.versionCode = 1

- ☐ `ios.bundleIdentifier` definido

Funcional

- ☐ App funciona sem erros no emulador
- ☐ Dados persistem ao fechar e reabrir
- ☐ Nenhum `console.log` com dados sensíveis

Loja

- ☐ Conta de desenvolvedor criada e aprovada
- ☐ Ficha da loja preenchida (título, descrição, screenshots, categoria)
- ☐ Classificação de conteúdo respondida

Referências

- [EAS Build — documentação oficial](https://docs.expo.dev/build/introduction/) (https://docs.expo.dev/build/introduction/)
- [Publicar na Google Play com EAS](https://docs.expo.dev/submit/android/) (https://docs.expo.dev/submit/android/)
- [OTA Updates com eas update](https://docs.expo.dev/eas-update/introduction/) (https://docs.expo.dev/eas-update/introduction/)
- [app.json — todas as propriedades](https://docs.expo.dev/versions/latest/config/app/) (https://docs.expo.dev/versions/latest/config/app/)
- [Google Play Console](https://play.google.com/console) (https://play.google.com/console)
- [Diretrizes de revisão da App Store](https://developer.apple.com/app-store/review/guidelines/) (https://developer.apple.com/app-store/review/guidelines/)